

Change request log

1. Concept Location

Step #	Description	Rationale
1	We ran the system	
2	We interacted with the system: after logging in we entered the schedule screen.	To get familiar with some of the features of the system and identify the areas where the values are being displayed. We found the values were getting displayed in watchlist and in the point details
3	We searched for "data" using the regular expression feature of the Visual Studio IDE's search tool.	We got lots of search results for the search and were unsure about how to identify the file to be edited.
4	We searched for "PointDetails" using the regular expression feature of the IDE's search tool.	we identified a button in the screen called "Point Details", so we tried searching using this keyword
5	We got around 46 results, which we explored	We found "BaseDwr.java" file. In this file we inspected the setPoint method, and we identified that this class allowed setting of a given data point. We noticed that this class might be helpful for truncating the values.
6	From the remaining results, we clicked on the file named DataPointDetailsDwr.java and this file had a class named "DataPointDetailsDwr".	We selected this class because while inspecting we were able to find that data was rendered from this class.
7	Then we noticed the "MiscDwr.java" and inspected it	The "MiscDwr.java" had a search result for "pointdetails" named "setPointDetails". It was just a setter method so we decided that it might not be of any use
8	We marked "DataPointsDetailsDwr" and "BaseDwr.java" as "located"	We confirmed that these were the classes to be modified as we were able to see the basic changes reflected.

Time spent (in minutes): 90 minutes

Classes and methods inspected:

- \mangoSource\build\resources\BaseDwr.java
 - int setpoint method
- \mangoSource\src\com\serotonin\mango\web\dwr\DataPointDetailsDwr.java
 - DataPointDetailsDwr class
- \mangoSource\src\com\serotonin\mango\web\dwr\MiscDwr.java

2. Impact Analysis

Step #	Description	Rationale
1	We made a list of methods called by Impact classes	To track the classes that could be impacted by the change.
2	We inspected the class "BaseDwr". Such class was marked as "to change" as well	We realized that this class had to be changed because it provides methods for preparing and managing state information related to data points, events, user comments, setting data points, forcing point reads, and other related tasks. Additionally, it includes methods

		for generating content, handling localization, and dealing with date ranges. We also noticed that this class is designed to be extended by subclasses that implement specific functionality for web remoting.
3	Within the BaseDwr class we inspected the “setChange” method and set it to “not change”.	We examined this method assuming that it would be useful to change values of data points, but we found out that the setChange method is responsible for setting the change information for a given data point. So, we marked it to not be changed.
4	We also examined the “setPoint” method in the “BaseDwr” class and set it to be changed	The "setPoint" method was examined and potentially modified due to the need for allowing the setting of a specific data point. This method is crucial for functionality utilized by the watch list and point details pages within the Mango system.
5	We also examined the “setPrettyText” method in the “BaseDwr” class and set it to be changed	We examined this method because it deals with the representation of a data point's value in a state object.
6	We inspected the class “DataPointDetailsDwr”. Such class was marked as "to change" as well	We analyzed this class because it extended BaseDwr.and we understood that this class handles dynamic web remoting functionality related to displaying and retrieving data for a specific data point.
7	We then investigated the “getDataPoints” method and set it to be changed	This method works with the retrieval of current state and information regarding the data point and setting pretty texts. We realized this method must be changed to reflect the change on the web page.
8	We examined other methods like “getHistoryTableData”, “getImageChartData”, “getChartData” and “getStatsChartData”. We marked all these methods as to be changed.	All the mentioned methods were examined, and we found that all these methods share a common theme of retrieving data related to a data point for specific purposes. Later when we implemented the previous changes, we were able to achieve the desired results even before we could make changes in these methods. so, we didn’t make any changes to these methods though we considered it to have high impact on the results.
9	We reviewed the “doLongPoll” method within the MiscDwr class and marked it as to not be changed	We examined this method because our IDE search result for “point details” returned this class as one of the results. And when we examined this class thoroughly, we found that there was a line of code “state.setPointDetailsState(newState)” within the doLongPoll method.It sets the pointDetailsState in the state object to the new state (newState). This is done to keep track of the latest state for future comparisons. We understood that this doesn’t have anything to do with the change we were working on. So, we decided to pass this class.

Time spent (in minutes): 120 minutes

Classes and Methods inspected

- \mangoSource\build\resources\BaseDwr.java
 - BaseDwr class
 - setpoint method
 - setChange method
 - setPrettyText method
- \mangoSource\src\com\serotonin\mango\web\dwr\DataPointDetailsDwr.java
 - DataPointDetailsDwr class
 - getDataPoints method

- getChartData method
- getStatsChartData method
- getHistoryTableData method
- getImageChartData method
- \mangoSource\src\com\serotonin\mango\web\dwr\MiscDwr.java
 - MiscDwr class
 - doLongPoll method

3. Prefactoring (optional)

Step #	Description	Rationale
1	Analyzed the existing code and identified potential improvements	Reviewed the codebase to identify areas for possible refactoring and improvement.
2	Conducted a code walkthrough to identify refactoring opportunities	Inspected the BaseDwr class and related classes/methods (DatapointsDetailsDwr.java) to identify potential improvements in code structure and design.
3	Identified a method with complex logic and low cohesion	Found a method in the BaseDwr class with complex logic and low cohesion that could be refactored for better readability and maintainability.
4	Committed the refactoring changes with git	Documented the refactoring changes in version control for future reference and collaboration with other team members.
50	Noted the time spent on the prefactoring process	Documented the time spent to track the effort involved in the prefactoring process.

Time spent (in minutes): 50 minutes

Classes/Methods Inspected:

- BaseDwr
- Identified method with complex logic

4. Actualization

Step #	Description	Rationale
1	In order to truncate the data provide in the watchlist to have two decimal points we examined BaseDwr class in "BaseDwr.java" file	We tried changing the setpoint function, but it threw errors and we were not able to successfully build the software .
2	We then examined the other methods available in the class and we came across the setPretty. We figured that ".setValue" was used in this function, so we decided to change this code.	In this method the point value was converted to html format and was set as the sensor value in the watch list. We added a few lines of code that will take a point value that is not null, truncate the extra decimal points , convert it to string format and set it to the current watchlist state.
3	We inspected the "BaseDwr.java" file further to find any other methods to be changed	We looked into the setChart method, but we couldn't find anything that has to be changed so we decide to move to the next file.

4	We notice that the DataPointDetailsDwr class in the DataPointDetailsDwr.java file extends the BaseDwr class. So, we looked into it.	We found that there was “rendered data” in the “getHistoryTableData” method within the “DataPointDetailsDwr” class and we tried modifying it to see if any changes are being reflected in the historical data. However, we observed that the alterations were not taking effect. This was attributed to the fact that the rendered data in question was intricately connected with the underlying data itself.
5	We then tried changing the “raw data”	We made some basic changes like setting the value to be negative to the value to check if the changes made to the values were reflected in the software. After building we were able to see the changes reflected in the software, so we decided to perform the truncation of historical values within this method.
6	We changed the code in line 94, where the .setValue operation was performed.	We understood that this line was used to set the sensor values from html formatted text. We changed this line of code to take the value as a “double” and we used the “%.2f” to truncate it to two decimal points. We then convert the number to string format to set the historical value of the sensor.
7	After performing these changes values displayed everywhere in the software were truncated to have two decimal points. But while clicking point details we had an error that stated that "minimum cannot be resolved to a variable" and "maximum cannot be resolved to a variable"	This error indicated that the variables minimum and maximum are not defined in the scope of the code where they are being used. To resolve this issue, we had to ensure that these variables are defined or passed appropriately to the JSP page. So, we inspected and analyzed the StatsChart.jsp server file which was responsible for rendering the page with error. We modified the code using Expression Language expressions, but we were not able to clear out the error message. We altered the code to use the <c:set> tag from JSTL to create truncated variables for minimum, maximum, and average. The fn:substring and fn:length functions were then used to achieve the desired truncation without error message.

Time spent (in minutes): 180 minutes

Classes and methods:

Inspected

- \mangoSource\build\resources\BaseDwr.java
 - BaseDwr class
 - setPoint - method
 - setPrettyText -method
 - setChart - method
- \mangoSource\src\com\serotonin\mango\web\dwr\DataPointDetailsDwr.java
 - DataPointDetailsDwr class
 - getHistoryTableData – method

- \mangoSource\build\WEB-INF\snippet\statsChart.jsp
- \mangoSource\build\WEB-INF\snippet\common.jsp

Changed

- \mangoSource\build\resources\BaseDwr.java
 - BaseDwr class
 - setPrettyText -method
- \mangoSource\src\com\serotonin\mango\web\dwr\DataPointDetailsDwr.java
 - DataPointDetailsDwr class
 - getHistoryTableData – method
- \mangoSource\build\WEB-INF\snippet\common.jsp

5. Postfactoring (optional)

Step #	Description	Rationale
1	Removed unnecessary changes in datapointsdetailsdwr.java	Upon analyzing the test results, we found that only the first change related to truncation was necessary.
2	Restored the original code in datapointsdetailsdwr.java	Reverted the changes made in datapointsdetailsdwr.java to the original state before the unnecessary modifications.
3	Re-ran unit tests and system tests for Schedule class	Ensured that reverting the changes did not introduce any regression and that the Schedule class functioned as expected.
4	Conducted manual testing on the schedule screen	Verified that the schedule screen functionality remained intact after reverting the unnecessary code changes.
5	Reverted changes were committed and pushed to the repository	Documented the reversal of changes in version control for future reference and to maintain code integrity.

Time spent (in minutes): 100 minutes

Changed files:

BaseDwr.java

DatapointsDetailsDwr.java

6. Validation

Step #	Description	Rationale	Result
1	Unit testing: Set up a data point with a decimal value	Ensure there is a data point with a decimal value for testing	Pass
2	Apply only the first change in datapointsdetailsdwr.java	Apply the code change related to truncation in datapointsdetailsdwr.java alone	Fail
3	Verify that only the second change is sufficient in Basedwr.java	Confirm that the display of data point values is correct with only the first change	Pass (This helped us to analyse that only one change is enough, so we rolled back the other change)
4	System Testing: Check the display of sensor values in the watch list	Verify that sensor values are displayed with at most two decimal places and truncated	Pass
5	Verify behavior with sensor values having no decimal places	Test with sensors having integer values and ensure correct display	Pass

6	Regression Testing: Confirm that rounding does not occur unintentionally	Verify point details display with and without code changes	Pass
7	Apply code changes to point details display	Ensure the code changes are applied to the "point details" interface	Pass
8	Integration Testing: Integrate code changes into the watch list module	Verify seamless integration with the watch list module	Pass
9	Confirm correct display in different Mango interfaces through UI	Ensure that the changes are consistently applied across various Mango interfaces	Pass
10	Acceptance Testing: Confirm that values are truncated, not rounded	Apply the code changes and check that values are indeed truncated as expected	Pass

Time spent (in minutes): 70 minutes

7. Summary of the change request

Phase	Time (minutes)	No. of classes inspected	No. of classes changed	No. of methods inspected	No. of methods changes
Concept location	90	3	2	1	1
Impact Analysis	120	3	2	9	2
Prefactoring	50	3	2	9	2
Actualization	180	2	2	4	2
Postfactoring	100	2	2	2	2
Verification	70	3	2	9	2
Total	610	3	2	9	11

8. Conclusions

This change request presented considerable challenges, particularly in the initial stages, where concept location proved to be a demanding task. The system's complexity and unfamiliarity posed hurdles during the identification of relevant classes and methods. Transitioning to the actualization phase, attempts to modify the setPoint function in the BaseDwr class encountered build errors, prompting a shift to alternative methods. The exploration of the setPretty method offered a solution, but unforeseen errors emerged during point details access, necessitating a detailed examination of JSP files. The incorporation of Expression Language and JSTL features was pivotal in resolving these errors and achieving the desired truncation of data. This initial change request highlighted the importance of adaptability and persistence, especially when dealing with complex software systems and navigating unforeseen challenges in the modification process.